

Capítulo 6 — Recursión

Una regla por profundidad

abuelo/2



bisabuelo/2



tatarabuelo/2



antepasado/2



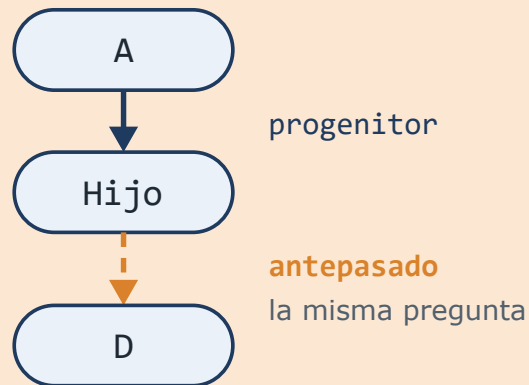
antepasado/2

```
antepasado(A, D) :-  
    progenitor(A, D).  
antepasado(A, D) :-  
    progenitor(A, Hijo),  
    antepasado(Hijo, D).  
  
?- antepasado(tare, isaac).  
true ;  
false.
```

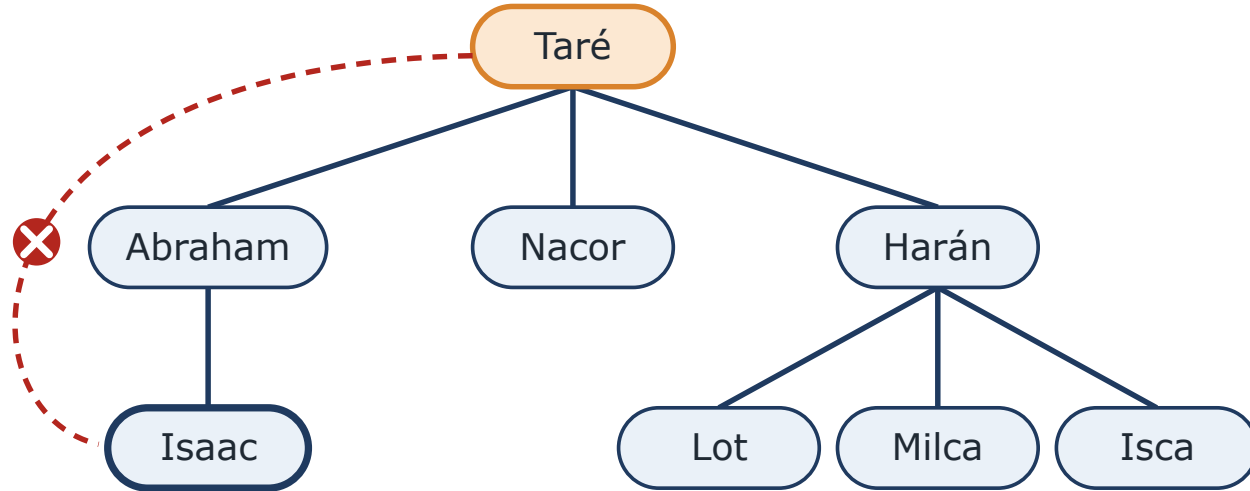
caso base



caso recursivo



Una generación por llamada · 1



`antepasado(tare, isaac)`

caso base

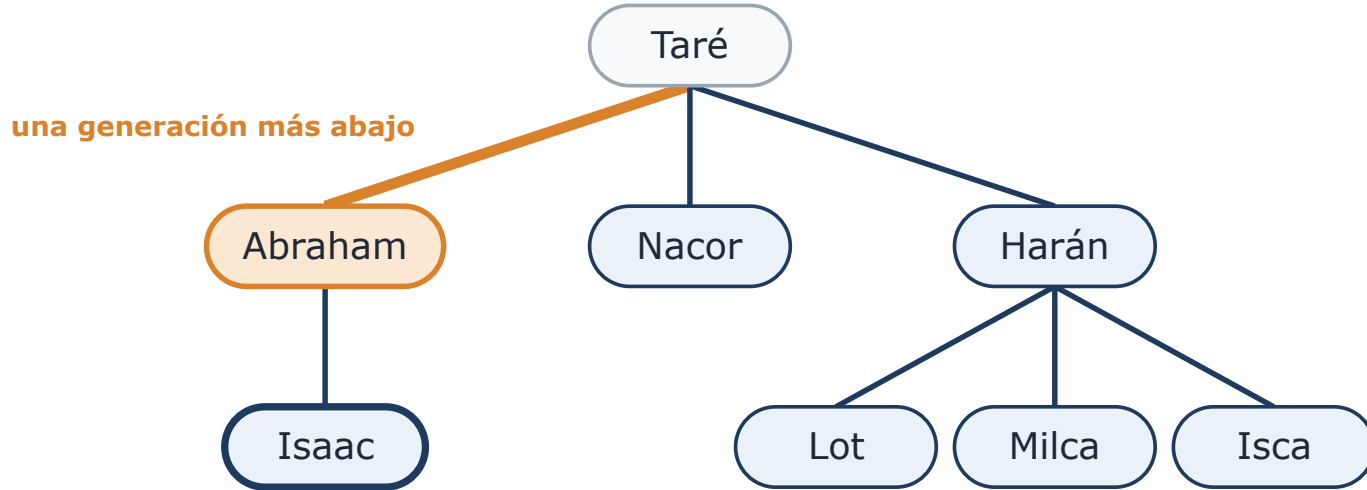
`progenitor(tare, isaac)`

falla

caso recursivo

se intenta a continuación

Una generación por llamada · 2



`antepasado(tare, isaac)`

caso recursivo

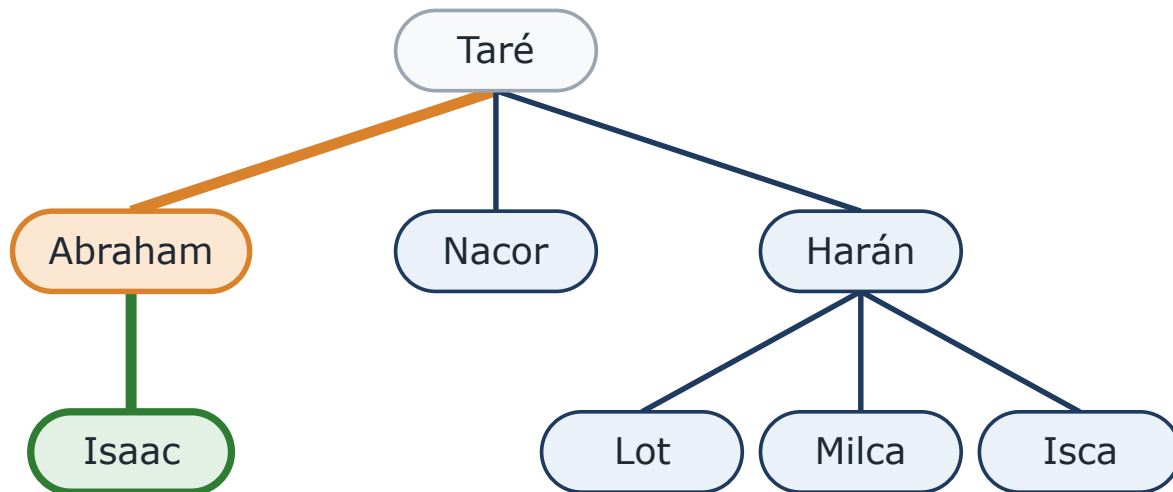
`progenitor(tare, Hijo)`

Hijo = abraham

nueva llamada

`antepasado(abraham, isaac)`

Una generación por llamada - 3



`antepasado(abraham, isaac)`

caso base

`progenitor(abraham, isaac)`

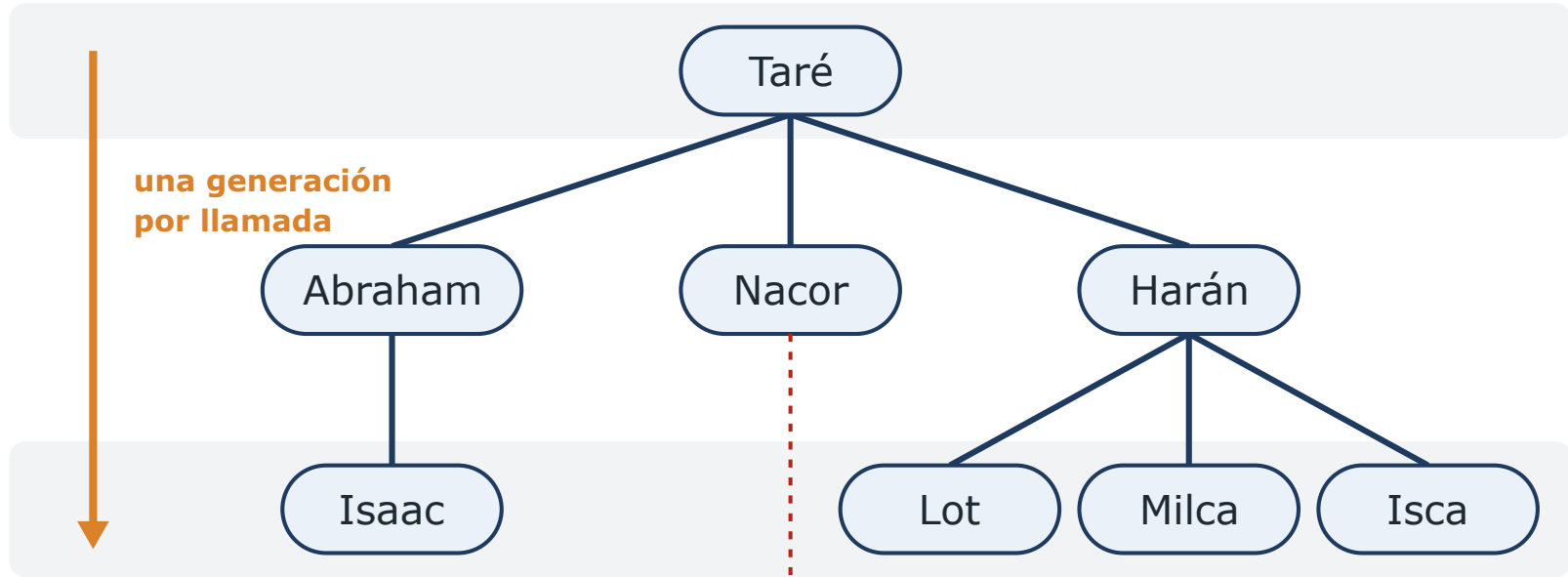
se prueba



true

también para `antepasado(tare, isaac)`

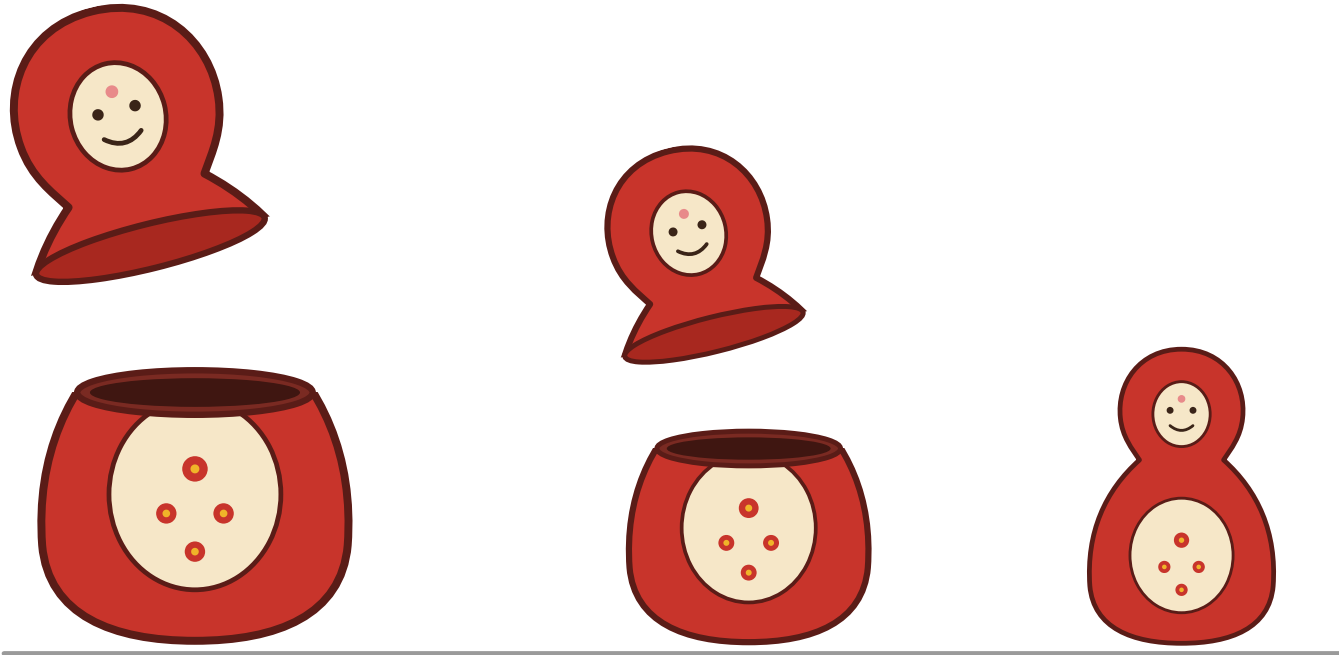
Por qué termina



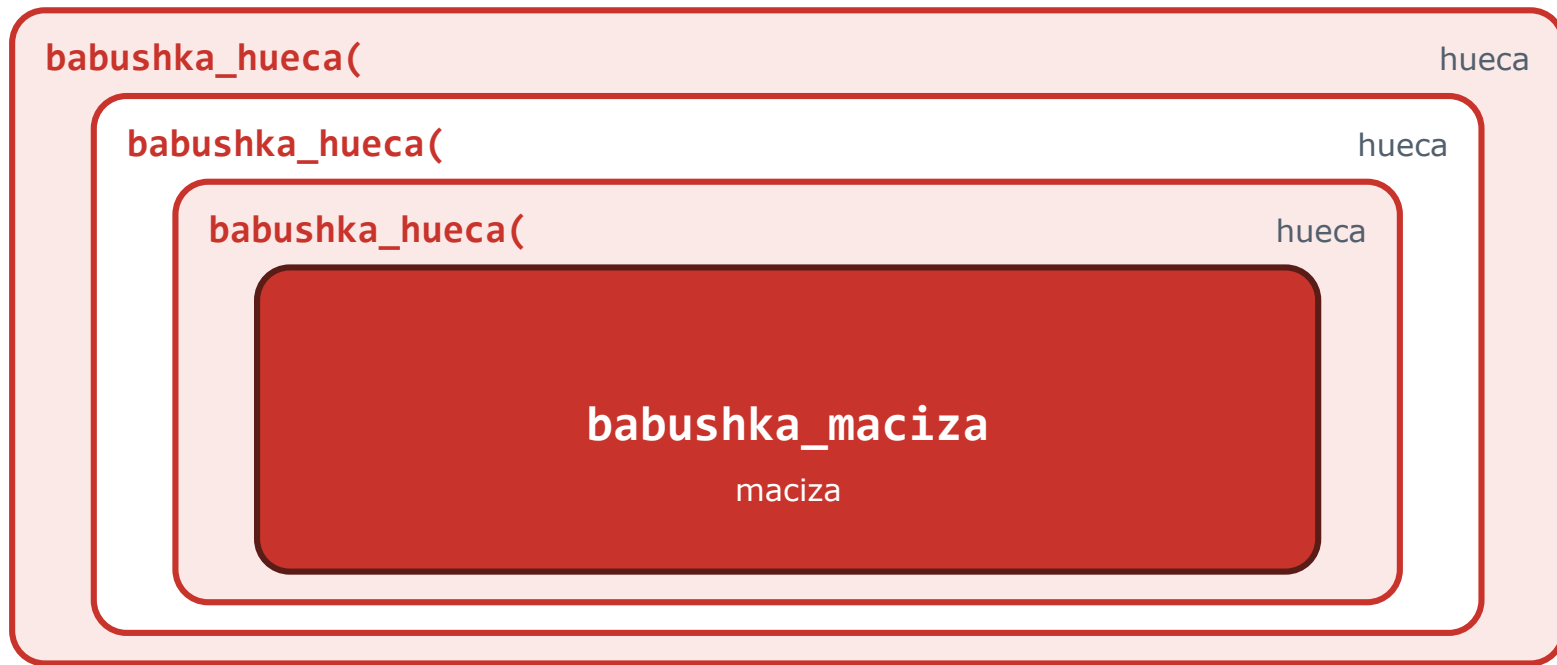
sin hijos: progenitor(A, Hijo) falla

la rama se agota

La babushka



Una babushka es un término



```
babushka_hueca(babushka_hueca(babushka_hueca(babushka_maciza)))
```

desarma/1

Caso base

```
desarma(babushka_maciza).
```

Caso recursivo

```
desarma(babushka_hueca(Interior)) :-  
    desarma(Interior).
```

Una muñeca por llamada



`desarma(babushka_hueca(babushka_hueca(babushka_hueca(babushka_maciza))))`

caso recursivo: se quita la muñeca exterior



`desarma(babushka_hueca(babushka_hueca(babushka_maciza)))`

caso recursivo: se quita la muñeca exterior



`desarma(babushka_hueca(babushka_maciza))`

caso recursivo: se quita la muñeca exterior



`desarma(babushka_maciza)`

caso base: la muñeca maciza no se abre



true.

Plantilla 8

```
p(CasoMinimo) :-  
    solucion_directa.
```

caso base

la instancia más simple,
sin invocarse

```
p(CasoGeneral) :-  
    un_paso(CasoGeneral, CasoMenor),  
    p(CasoMenor).
```

caso recursivo

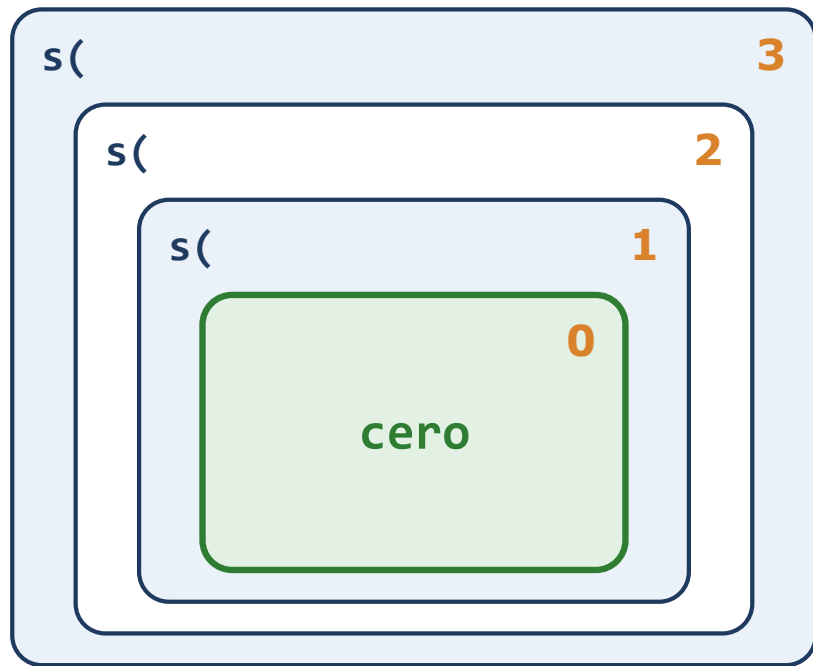
- 1 reduce el problema
- 2 se invoca sobre el caso menor

El caso base se escribe primero.

En el cuerpo, la reducción va antes que la llamada recursiva.

Los números naturales

```
%! natural(+N) is semidet.  
%! natural(-N) is multi.  
%  
% N es un número natural.  
natural(cero).  
natural(s(N)) :-  
    natural(N).
```



$s(s(s(cero)))$

Verificar y generar

Verifica

```
?- natural(s(s(cero))).  
true.
```

Genera

```
?- natural(N).  
N = cero ;  
N = s(cero) ;  
N = s(s(cero)) ;  
N = s(s(s(cero))) ;  
...
```

suma/3

```
%! suma(?A, ?B, ?C) is nondet.
```

```
%
```

```
% C es A más B.
```

```
suma(cero, B, B).
```

```
suma(s(A), B, s(C)) :-
```

```
    suma(A, B, C).
```

```
?- suma(s(cero), s(s(cero)), Cuanto).
```

```
Cuanto = s(s(s(cero))).
```

La suma, llamada por llamada · 1

`suma(s(cero), s(s(cero)), Cuanto)`

cláusula 2: una s menos en el primero

llamada recursiva

`suma(cero, s(s(cero)), C)`

cláusula 1: caso base

`Cuanto = s(C)`

?

La cabeza `suma(s(A), B, s(C))` unifica con `A = cero`:
el resultado queda escrito como `s(C)`.

La suma, llamada por llamada · 2

`suma(s(cero), s(s(cero)), Cuanto)`

cláusula 2: una s menos en el primero

`Cuanto = s(C)`

llamada recursiva

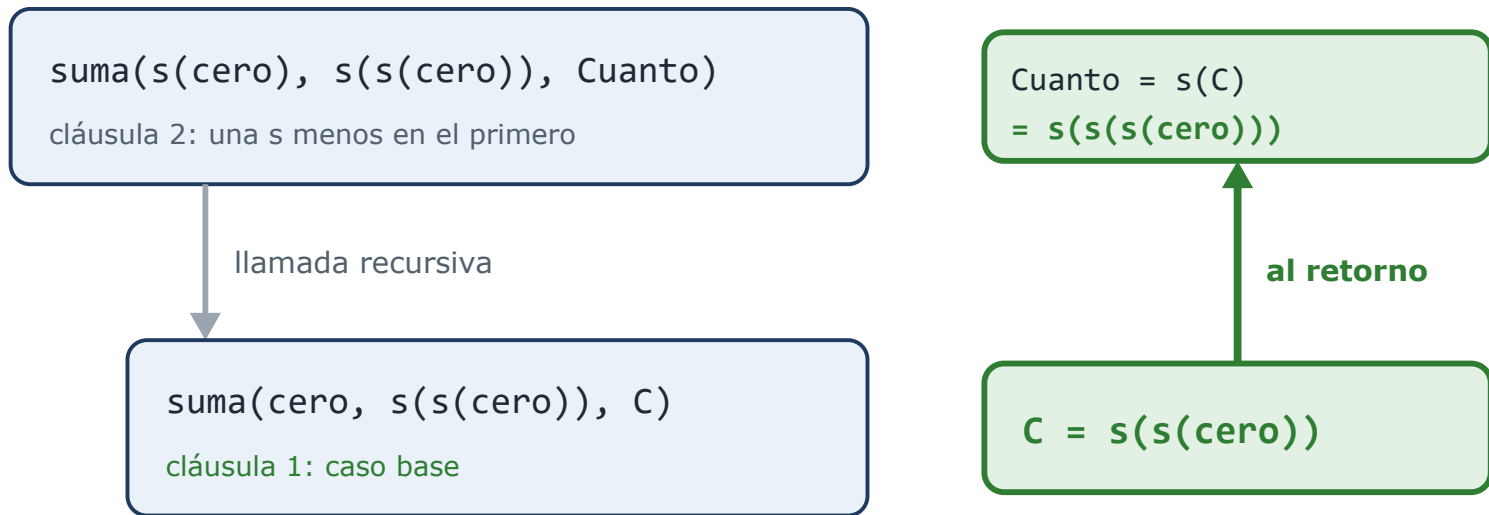
`suma(cero, s(s(cero)), C)`

cláusula 1: caso base

`C = s(s(cero))`

Con cero en el primer argumento se aplica el caso base:
el resultado es el segundo argumento.

La suma, llamada por llamada · 3



C ya tiene valor, y con él Cuanto:
uno más dos es tres.

Sumar y restar

```
?- suma(s(cero), s(s(cero)), Cuanto).
```

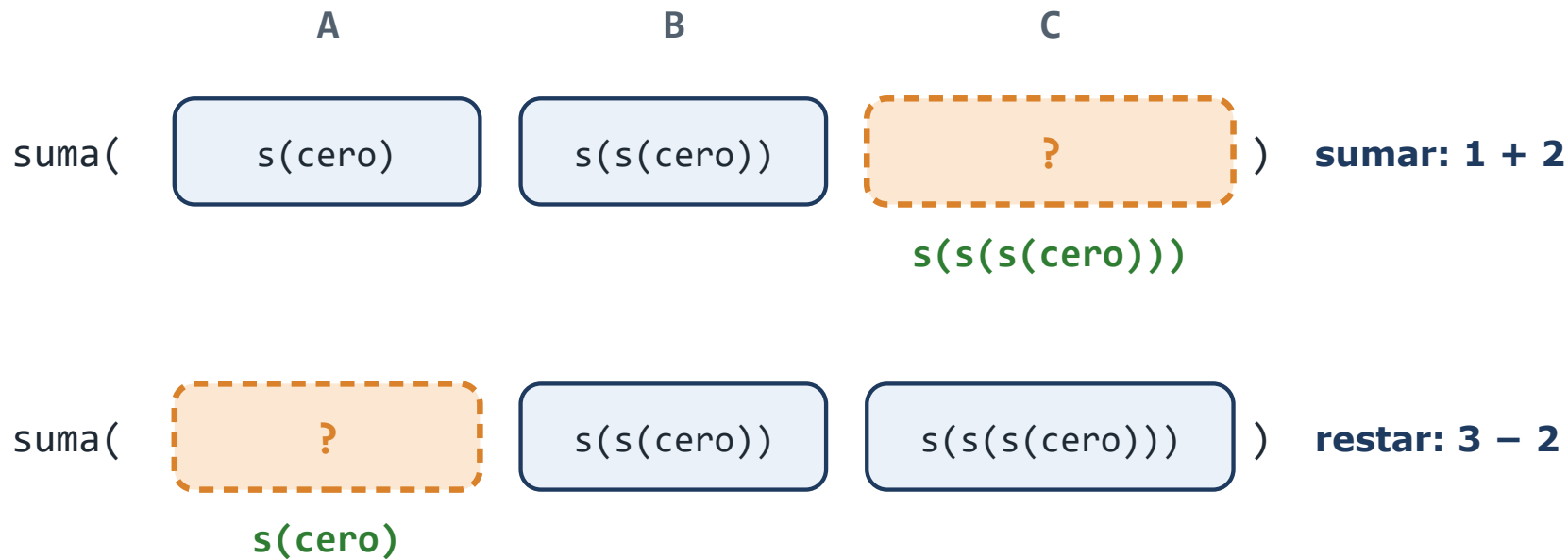
```
Cuanto = s(s(s(cero))).
```

```
?- suma(A, s(s(cero)), s(s(s(cero)))).
```

```
A = s(cero) ;
```

```
false.
```

Una relación entre tres números



La consulta decide cuáles son los datos y cuál es la incógnita.

¿Qué se reduce?

`suma(s(s(cero)), B, C)`

el primer argumento

`s(s(cero))`



`s(cero)`



`cero`

caso base



termina: dos llamadas

`natural(N)`, con `N` libre

los términos que construye

`cero`

`s(cero)`



`s(s(cero))`



`s(s(s(cero)))`



...



nada se reduce: no termina

generaciones/3

Caso base

```
generaciones(A, D, 1) :-  
    padre(A, D).
```

Caso recursivo

```
generaciones(A, D, N) :-  
    padre(A, Hijo),  
    generaciones(Hijo, D, Faltan),  
    N is Faltan + 1.
```

Dos consultas

```
?- generaciones(juan, eva, Cuantas).  
Cuantas = 3 ;  
false.
```

```
?- generaciones(juan, Quien, 2).  
Quien = luis ;  
false.
```

Las generaciones, al retorno



generaciones(juan, eva, N)

↓ ① padre(juan, ana)

generaciones(ana, eva, F1)

↓ ① padre(ana, luis)

generaciones(luis, eva, F2)

caso base: padre(luis, eva)

$N = F1 + 1 = 3$

↑ ③

$F1 = F2 + 1 = 2$

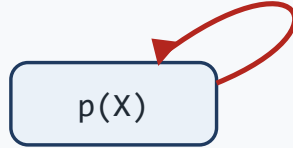
↑ ③

$F2 = 1$

- ① padre(A, Hijo) avanza una generación
- ② la llamada recursiva resuelve el resto y da Faltan
- ③ N is Faltan + 1 suma esta generación, al retorno

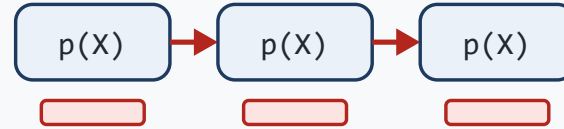
Cuatro causas de no terminación

1 Falta el caso base



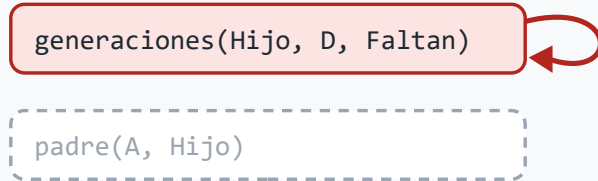
solo la cláusula recursiva:
ninguna llamada termina

2 No reduce el problema



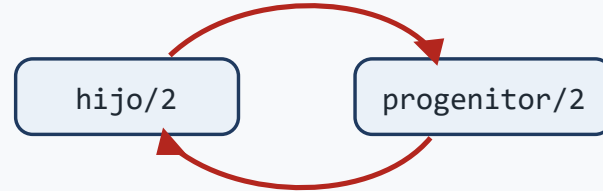
el mismo tamaño en cada llamada

3 Reduce después de llamar



nunca se alcanza

4 Dos predicados en ciclo



ninguno llega a un hecho

Ante un programa que no termina, se verifica en este orden: 1, 2, 3, 4.

La reducción, después

```
%! generaciones(?A, ?D, -N) is nondet.  
%  
% No termina: La llamada recursiva precede  
% al objetivo que reduce el problema.  
generaciones(A, D, N) :-  
    generaciones(Hijo, D, Faltan),  
    padre(A, Hijo),  
    N is Faltan + 1.
```

Un ciclo entre dos predicados

```
%! hijo(?H, ?P) is nondet.
```

```
%
```

```
% H es hijo de P.
```

```
hijo(H, P) :-  
    progenitor(P, H).
```

```
%! progenitor(?P, ?H) is nondet.
```

```
%
```

```
% P es el padre o la madre de H.
```

```
progenitor(P, H) :-  
    hijo(H, P).
```

Resumen

Caso base

la instancia más simple,
sin una nueva invocación

Caso recursivo

reduce el problema
y se invoca a sí mismo

$s(N)$

el sucesor de N :
un término, no un número

Terminación

una magnitud se reduce
en cada llamada, y el
caso base es alcanzable

Verificar y generar

el mismo predicado,
según qué argumentos
lleguen instanciados

Orden en el cuerpo

la reducción va antes
de la llamada recursiva

Créditos

- Ilustraciones: dibujadas para el curso, licencia MIT.
- Ejemplos: ejemplos/capitulo-06 del curso.